arXiv

License: arXiv.org perpetual non-exclusive license (https://info.arxiv.org/help/license/index.html#licenses-available)
arXiv:2607.19985v1 [cs.AI] 22 Jul 2026

Coordinating from Memory: Graph-Structured Experience Reuse for Multi-Agent Adaptation in Dynamic Manufacturing

Chengxiao Dai

School of Computer Science, University of Sydney, Sydney, Australia.

Zhanhui Lin

School of Computer Science, University of Sydney, Sydney, Australia.

Zhaokun Yan

School of Computation, Information and Technology, Technical University of Munich, Munich, Germany.

Youyang Ni

School of Life and Environmental Sciences, University of Sydney, Sydney, Australia.

Chenjun Lei

College of Business and Economics, Australian National University, Canberra, Australia.

Luyan Zhang

† [†*Corresponding author (zhang.luya@northeastern.edu).]

Khoury College of Computer Sciences, Northeastern University, Boston, USA.

Abstract

Dynamic manufacturing environments require multi-agent systems to coordinate effectively under frequent operational disturbances such as machine failures, urgent job arrivals, and processing time variations. Existing multi-agent reinforcement learning approaches treat each disturbance episode independently, discarding valuable coordination experience that could accelerate future adaptation. In this paper, we propose a Graph-Structured Experiential Memory (GSEM) framework for multi-agent coordination in dynamic manufacturing. The framework encodes historical coordination episodes as heterogeneous relational graphs that capture task dependencies, machine states, and inter-agent collaboration patterns. When a new disturbance occurs, a graph neural network-based retrieval mechanism identifies structurally similar past episodes, enabling experience-guided policy adaptation rather than learning from scratch. Experiments on dynamic flexible job-shop scheduling benchmarks with three disturbance types show that GSEM reduces makespan by 4.1%–10.0% and adaptation time by 33%–38% compared to the strongest memory-augmented baseline, with the advantage increasing under higher disturbance frequency. Ablation studies and cross-disturbance transfer experiments further validate the necessity of graph-structured encoding and similarity-based retrieval, and demonstrate the cross-disturbance generalizability of learned coordination patterns.

I INTRODUCTION

Modern manufacturing systems in the Industry 4.0 era are characterized by high product variety, short production cycles, and fre-

quent uncertainty, making shop-floor scheduling much more challenging than in traditional static settings [1]. In practice, production plans are often disrupted by machine breakdowns, urgent job arrivals, material shortages, and processing-time fluctuations, which may propagate across machines, jobs, and production stages [2]. Dynamic scheduling should therefore be viewed not simply as repeated local rescheduling, but as an online coordination problem in which distributed decision makers detect disturbances and adjust actions to maintain production performance [2, 3].

Multi-agent systems provide a natural foundation for distributed manufacturing control by decomposing decision-making across machines or production units while preserving system-level coordination objectives [3]. Based on this paradigm, reinforcement learning (RL) and multi-agent reinforcement learning (MARL) have shown strong potential for dynamic scheduling, especially in flexible job-shop environments [4, 5]. Recent studies have reported promising results using cooperative MARL methods in dynamic manufacturing scenarios [6, 7, 8]. However, most existing approaches still treat disturbance episodes as largely independent. When a new disruption occurs, agents usually rely on the current policy or replayed transitions, which mainly improve training efficiency rather than support structured adaptation during execution [9, 10]. As a result, current MARL methods often show slow adaptation, weak reuse of coordination experience, and limited generalization across heterogeneous disturbances [4, 6].

This limitation is important because disturbances in manufacturing

are rarely isolated. A local machine failure may trigger downstream waiting, queue imbalance, and coordination conflicts among multiple agents [2]. Effective adaptation depends on whether the system can recall and reuse coordination patterns from similar past situations, much as human operators handle disruptions by recalling similar incidents [11]. A key challenge is representation: manufacturing coordination is inherently relational, involving precedence constraints, resource competition, and inter-agent dependencies that are difficult to preserve using flat state-action vectors [9, 10]. Graph neural networks (GNNs) can model such relational structures effectively [12, 13, 14, 15, 16], but have mainly been used for current-state encoding rather than experience storage.

This paper proposes a *Graph-Structured Experiential Memory* (GSEM) framework that stores coordination episodes as heterogeneous relational graphs and retrieves structurally similar past experiences for rapid adaptation. The key insight is that what needs to be reused is not a raw past state, but a *reusable recovery motif*: the coordinated rerouting pattern after a breakdown, the queue rebalancing after an urgent job arrival, or the conflict resolution under processing time variation. These motifs are inherently relational, and flat episodic memory cannot preserve them.

The main contributions of this paper are:

- We propose graph-structured coordination memory for disturbance adaptation in multi-agent manufacturing, where past recovery episodes are stored as heterogeneous relational graphs that preserve the structural dependencies essential for experience reuse.

- We design a structure-aware memory retrieval mechanism that identifies relevant past episodes through GNN-based graph similarity with quality-recency weighting, rather than flat nearest-neighbor matching.

- We integrate retrieved experience into multi-agent coordination through gated attention fusion and memory-conditioned communication, enabling agents to collectively exploit past recovery motifs for faster adaptation.

II RELATED WORK

II-A *Multi-Agent Coordination in Manufacturing*

Multi-agent systems have been widely used in manufacturing scheduling and control because of their modularity, scalability, and fault tolerance [3]. Early studies mainly relied on rule-based coordination or contract-net-style negotiation, in which agents allocated tasks through predefined interaction protocols [17]. Although effective in stable environments, these approaches are less adaptable to disruptions such as machine failures, urgent job arrivals, and other dynamic events.

With the development of learning-based scheduling, RL has been increasingly applied to manufacturing decision-making. Single-agent RL methods typically formulate scheduling as a Markov decision process and learn a centralized dispatching policy for the shop floor [5]. However, they often face scalability challenges as the numbers of jobs, machines, and decisions increase. MARL distributes decision-making across multiple machines or production units [4]. Representative approaches include independent learning [18], value decomposition methods such as QMIX [7], and cooperative policy optimization methods such as Multi-Agent Proximal Policy Optimization (MAPPO) [8]. Recent studies have also shown the potential of adaptive multi-agent scheduling in dynamic manufacturing environments [6].

Most MARL-based scheduling methods focus on reacting to the current state rather than reusing coordination experience across disturbances. When disruptions occur, agents usually rely on learned policies or replay mechanisms, which may limit adaptation and generalization.

II-B *Memory-Augmented Learning*

Memory augmentation has been widely studied to improve the adaptability of neural decision systems. Early architectures such as Neural Turing Machines and Differentiable Neural Computers introduced differentiable read-write mechanisms that allow models to store and retrieve information beyond fixed network parameters [19, 20]. In reinforcement learning, experience replay and prioritized replay are standard techniques for improving sample

efficiency by reusing past transitions during training [9, 10].

More recently, episodic memory mechanisms have been proposed to support faster adaptation by storing and retrieving past episodes rather than isolated transitions [21, 22]. These methods typically retrieve historical experiences through nearest-neighbor matching in an embedding space, allowing agents to recall useful behaviors in similar situations.

However, these methods store experience as fixed-dimensional vectors or flat transition sequences, which cannot preserve the relational dependencies among jobs, operations, machines, and agents that are intrinsic to manufacturing coordination.

II-C *Graph Neural Networks in Manufacturing*

Graph neural networks have emerged as an effective tool for modeling relational structures in manufacturing and scheduling problems. Job-shop and flexible job-shop scheduling naturally involve precedence constraints, machine-operation compatibility, and resource conflicts, making graph-based representations suitable for learning scheduling policies [12]. Several studies have used GNNs to encode scheduling states and learn dispatching decisions, showing strong performance and generalization across problem instances [13, 14]. Heterogeneous graph neural networks further improve representational expressiveness by distinguishing different node and edge types, such as jobs, machines, operations, and resource relations [15].

In multi-agent settings, graph-based learning has also been used to model communication and coordination structures among agents [16]. By propagating information along relational edges, GNNs allow agents to reason about neighboring states and interactions more effectively than flat feature encodings.

Although graph-based methods capture relational structure effectively, they are mainly used to encode the current system state for reactive decision-making; their use as memory units for storing and retrieving historical coordination episodes has received much less attention.

III PROPOSED METHOD

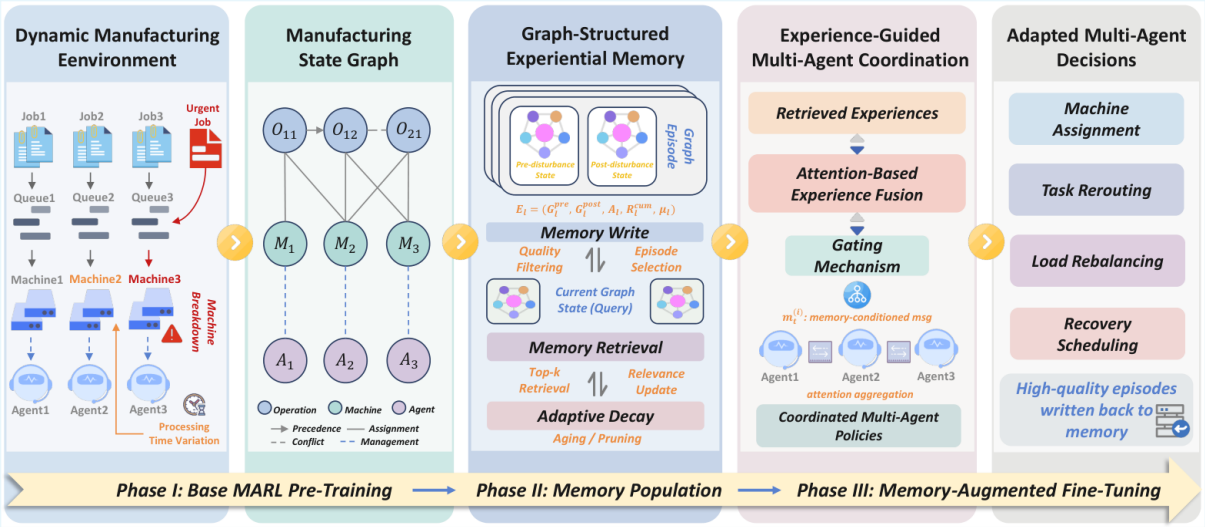


Fig. 1: Overall framework of the proposed GSEM system.

The goal of GSEM is not only to improve current-state encoding, but to store and retrieve reusable coordination structures from past disturbance recovery episodes. We consider a Dynamic Flexible Job-Shop Scheduling Problem (DFJSP) with n jobs $\mathcal{J}$ and m machines $\mathcal{M}$, where each job consists of ordered operations with machine-dependent processing times. Dynamic disturbances, including machine breakdowns, urgent job arrivals, and processing time variations, occur stochastically during execution. We model this as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP) with N scheduling agents, each managing a disjoint subset of machines. The goal is to find a joint policy $\boldsymbol{\pi}^*$ that minimizes the expected makespan $C_{\max}$ under disturbances. At each step t, each agent receives a shared reward composed of a progress term and a completion penalty:

$$r_t = -\alpha \cdot \Delta C_{\max}(t) - \beta \cdot \sum_{j \in \mathcal{J}} \max(0,\, C_j - d_j) \tag{1}$$

where $\Delta C_{\max}(t)$ is the incremental makespan change, C_j is the completion time of job j, d_j is its due date, and α, β are weighting coefficients.

III-A *Manufacturing State Graph*

We represent the manufacturing state at each decision point as a heterogeneous graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ with three node types (operation, machine, and agent) and four edge types: *precedence* (operation sequencing within jobs), *assignment* (operation-to-compatible-machine links weighted by processing time), *management* (machine-to-agent ownership), and *conflict* (operations competing for the same machine). A Relational Graph Convolutional Network

(R-GCN) encoder produces node embeddings through L message-passing layers, and a **type-aware readout** generates a graph-level embedding by mean-pooling each node type separately and concatenating:

$$\mathbf{z}_{\mathcal{G}} = \Big[\overline{\mathbf{h}}^{\text{op}} \| \overline{\mathbf{h}}^{\text{mc}} \| \overline{\mathbf{h}}^{\text{ag}}\Big], \quad \overline{\mathbf{h}}^{\tau} = \frac{1}{|\mathcal{V}^{\tau}|} \sum_{v \in \mathcal{V}^{\tau}} \mathbf{h}_v^{(L)} \tag{2}$$

This preserves the distinct semantics of different node types that would be lost in naive global pooling.

III-B *Graph-Structured Experiential Memory*

The memory pool $\mathcal{P}$ stores bounded past coordination episodes. Each entry is a tuple $E_l = (\mathcal{G}_l^{\text{pre}}, \mathcal{G}_l^{\text{post}}, \mathbf{A}_l, R_l^{\text{cum}}, \boldsymbol{\mu}_l)$ containing the state graphs before and after a disturbance, the joint action sequence during recovery, the cumulative reward, and metadata (disturbance type, severity, timestamp).

Memory Write. Episodes are filtered by two criteria before storage. First, episode boundaries are identified by detecting disturbance onset (a $|\Delta C_{\max}|$ spike beyond 2σ of recent history) and recovery completion (per-step reward returning within 1σ of its pre-disturbance moving average). Second, only episodes whose cumulative reward satisfies $R_l^{\text{cum}} \geq \overline{R} - \kappa \cdot \sigma_R$ are retained, where $\overline{R}$ and σ_R are running statistics and κ controls selectivity.

Memory Retrieval. When a disturbance occurs, the current state graph $\mathcal{G}_t$ is encoded and compared against stored entries using

cosine similarity with a quality-recency weight:

$$\mathrm{sim}(\mathcal{G}_t, E_l) = \frac{\mathbf{z}_{\mathcal{G}_t}^{\top}\, \mathbf{z}_{\mathcal{G}_l^{\mathrm{pre}}}}{\|\mathbf{z}_{\mathcal{G}_t}\| \cdot \|\mathbf{z}_{\mathcal{G}_l^{\mathrm{pre}}}\|} \cdot \widetilde{R}_l \cdot e^{-\eta(t-\tau_l)} \tag{3}$$

where $\widetilde{R}_l = (R_l^{\mathrm{cum}} - R_{\min})/(R_{\max} - R_{\min}) \in [0,1]$ is a min-max-normalized quality score (needed because cumulative rewards are non-positive under Eq. (1)), and $R_{\min}, R_{\max}$ are running statistics over $\mathcal{P}$. Top-K episodes are then selected via maximal marginal relevance (MMR) to balance relevance and diversity.

Adaptive Decay. Each entry maintains a relevance score ρ_l updated as:

$$\rho_l \leftarrow \rho_l \cdot \exp\!\Big(-\eta_d (t - t_l^{\mathrm{last}})\Big) + \mathbb{1}[E_l \in \mathcal{S}_t] \cdot \Delta\rho \tag{4}$$

where $\mathcal{S}_t$ is the top-K retrieved set at step t: retrieved entries receive a bonus $\Delta\rho$ and unused entries decay over time. When $|\mathcal{P}|$ exceeds capacity, the lowest-ρ entry is evicted.

III-C *Experience-Guided Coordination*

Retrieved episodes are integrated into each agent's policy through attention-based fusion and gating. For each retrieved episode E_l, the system exploits all stored components: the pre-disturbance graph $\mathcal{G}_l^{\mathrm{pre}}$ provides situational context, the post-recovery graph $\mathcal{G}_l^{\mathrm{post}}$ encodes the target coordination state, and the action sequence $\mathbf{A}_l$ supplies the recovery strategy. These are fused into a unified episode representation:

$$\mathbf{e}_l = \mathbf{W}_e\left[\mathbf{z}_{\mathcal{G}_l^{\text{pre}}} \| \mathbf{z}_{\mathcal{G}_l^{\text{post}}} \| f_a(\mathbf{A}_l)\right] + \mathbf{b}_e \tag{5}$$

where $f_a(\cdot)$ is a temporal mean-pooling over the action sequence embeddings.

Attention Fusion. Each agent i's R-GCN embedding $\mathbf{h}_{v_i}^{(L)}$ serves as query, while the episode representations $\{\mathbf{e}_l\}_{l=1}^{K}$ serve as keys and values via cross-attention with learned projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d}$:

$$\begin{aligned} \alpha_l^{(i)} &= \text{softmax}\left(\frac{(\mathbf{W}_Q \mathbf{h}_{v_i}^{(L)})^\top (\mathbf{W}_K \mathbf{e}_l)}{\sqrt{d}}\right), \\ \mathbf{c}_t^{(i)} &= \sum_{l=1}^{K} \alpha_l^{(i)} \mathbf{W}_V \mathbf{e}_l \end{aligned} \tag{6}$$

Gating Mechanism. A learned gate $\mathbf{g}_t^{(i)}$ adaptively balances between current state and retrieved experience:

$$\mathbf{g}_t^{(i)} = \sigma\left(\mathbf{W}_g[\mathbf{h}_{v_i}^{(L)} \| \mathbf{c}_t^{(i)}] + \mathbf{b}_g\right) \tag{7}$$

$$\widetilde{\mathbf{h}}_t^{(i)} = \mathbf{g}_t^{(i)} \odot \mathbf{c}_t^{(i)} + (1 - \mathbf{g}_t^{(i)}) \odot \mathbf{h}_{v_i}^{(L)} \tag{8}$$

The gate tends toward experience reliance immediately after a disturbance and shifts toward the current policy as the agent adapts.

Memory-Conditioned Communication. To enable collective experience-driven coordination, each agent i broadcasts a memory-

conditioned message $\mathbf{m}_t^{(i)} = \mathbf{W}_m \widetilde{\mathbf{h}}_t^{(i)}$ and aggregates incoming messages via attention over its neighbor set $\mathcal{N}(i)$, defined as agents whose machines share at least one operation with those of i via *assignment* or *conflict* edges in $\mathcal{G}_t$:

$$\begin{aligned} \beta_{ij} &= \mathrm{softmax}\left(\frac{(\widetilde{\mathbf{h}}_t^{(i)})^{\top} \mathbf{m}_t^{(j)}}{\sqrt{d}} \right) \\ \overline{\mathbf{m}}_t^{(i)} &= \sum_{j \in \mathcal{N}(i)} \beta_{ij} \mathbf{m}_t^{(j)} \end{aligned} \tag{9}$$

The final policy $\pi_i(a_t^{(i)} | \cdot)$ takes the concatenation $[\widetilde{\mathbf{h}}_t^{(i)} \| \overline{\mathbf{m}}_t^{(i)}]$ as input.

III-D *Training Procedure*

Training proceeds in three phases: (I) base policy pre-training with MAPPO [8] without memory; (II) memory population by rolling out the Phase I policy, with $\overline{R}, \sigma_R$ in the quality filter initialized from the Phase I reward distribution; (III) end-to-end fine-tuning with memory retrieval active and $\overline{R}, \sigma_R$ continuously updated. In Phase III, the R-GCN encoder is additionally trained with a contrastive loss that pulls graphs of the same disturbance type together, where $\mathcal{G}^+$ is sampled within ± 5 recovery steps of $\mathcal{G}_t$ and $\{\mathcal{G}_l\}_{l=1}^{B}$ are negatives from the current mini-batch:

$$L_{\mathrm{cont}} = -\log \frac{\exp(\mathrm{sim}(\mathbf{z}_{\mathcal{G}_t}, \mathbf{z}_{\mathcal{G}^+})/\tau)}{\sum_{l=1}^{B} \exp(\mathrm{sim}(\mathbf{z}_{\mathcal{G}_t}, \mathbf{z}_{\mathcal{G}_l})/\tau)} \tag{10}$$

During online adaptation, the agent detects disturbances, re-

trieves top-K episodes via Eqs. (3)–(4), applies experience fusion via Eqs. (5)–(9), and writes high-quality recovery episodes back to $\mathcal{P}$. Parameters are updated with $L_{\text{MAPPO}} + c_2 L_{\text{cont}}$.

IV EXPERIMENTS

IV-A *Experimental Setup*

We primarily evaluate GSEM on DFJSP instances with 10 jobs, 10 machines, and 5 scheduling agents, a scale consistent with widely used DFJSP benchmarks in the literature [12, 13, 14]. Each job consists of 3 to 6 operations, each compatible with 2 to 4 machines, with processing times sampled from $\mathcal{U}(1, 20)$. Three types of dynamic disturbances are injected during execution: machine breakdowns with failure rate $\lambda = 0.03$ per step and downtime drawn from $\mathcal{U}(5, 20)$, urgent job arrivals at rate 0.05 per step, and processing time variations with perturbation factor $\delta \sim \mathcal{U}(0.8, 1.2)$. Three disturbance frequency levels are tested: Low, Medium, and High, with 2–3, 5–7, and 10+ events per episode. *Adapt.* counts steps from disturbance onset until per-step reward recovers to within 1σ of its pre-disturbance moving average.

We compare against shortest processing time (SPT) and earliest due date (EDD) dispatching rules, a genetic algorithm (GA) with reactive rescheduling, centralized proximal policy optimization (PPO), and MARL methods including QMIX [7], MAPPO [8], and MAPPO+ER with flat experience replay. We further include two adapted baselines: GNN-Dispatch, a GNN-based reactive scheduler adapted from [13] without memory, and MAPPO+Epi, which

augments MAPPO with flat episodic memory following [22]. The R-GCN encoder has $L = 3$ layers with 128-dimensional hidden features, and the policy network is a 2-layer multi-layer perceptron (MLP) with 256 units. We train with Adam at learning rate 3×10^{-4}, discount factor $\gamma = 0.99$, and PPO clipping $\epsilon = 0.2$ for 20K Phase I and 30K Phase II+III episodes. The memory pool capacity is $|\mathcal{P}|_{\max} = 500$ with retrieval size $K = 5$. All results are averaged over 20 independent seeds on a server with dual AMD EPYC 9654 CPUs, 2 TB of DDR5 RAM, and $4\times$ NVIDIA RTX 5090 GPUs. The contrastive loss weight is set to $c_2 = 0.1$, the decay rate to $\eta_d = 0.01$, and the quality threshold parameter to $\kappa = 1.0$. The reward coefficients are $\alpha = 1.0$ and $\beta = 0.5$.

TABLE I: Performance Comparison on DFJSP (10×10, 5 Agents). Makespan ($C_{\max}$), Adaptation Speed (Adapt., steps), and Machine Utilization (U).

Method	Single Disturbance (Medium Freq.)						Mi
	Breakdown		Urgent Job		Time Variation		Low
	$C_{\max}$	Adapt.	$C_{\max}$	Adapt.	$C_{\max}$	Adapt.	$C_{\max}$
SPT	312 ±19	–	298 ±14	–	267 ±13	–	285 ±15
EDD	325 ±20	–	285 ±15	–	274 ±12	–	291 ±15
GA	268 ±13	42±9	255 ±13	38±6	241 ±11	35±7	248 ±12
PPO	259 ±15	35±8	248 ±12	32±9	234 ±10	28±6	241 ±14
QMIX	251 ±15	31±6	242 ±10	29±7	229 ±10	25±5	235 ±10
MAPPO	243 ±12	28±7	235 ±11	26±5	223 ±9	22±5	228 ±11
MAPPO+ER	238 ±10	25±5	231 ±10	24±4	220 ±7	20±4	224 ±10
GNN-Disp.[13]	245 ±12	27±7	237 ±10	25±6	221 ±8	21±5	230 ±11
MAPPO+Epi[22]	233 ±11	22±4	227 ±9	21±4	217 ±8	18±4	220 ±9
GSEM (Ours)	**218 ±9**	**14±4**	**214 ±8**	**13±3**	**208 ±7**	**12±3**	**209 ±10**

IV-B *Main Results*

Table I summarizes the results across all scenarios. Two baseline comparisons are particularly informative. First, GNN-Dispatch achieves performance comparable to MAPPO despite using a graph neural network encoder, confirming that GNN-based encoding of the *current* state alone does not improve adaptation to dynamic disturbances. Second, MAPPO+Epi benefits from episodic retrieval and outperforms MAPPO+ER, yet its reliance on flat state vectors limits its ability to preserve the relational recovery structures within coordination episodes. GSEM closes this gap by storing recovery motifs as graphs, consistently achieving the best performance with makespan reductions of 4.1%–6.4% and adaptation speed improvements of 33%–38% relative to MAPPO+Epi across all single-disturbance scenarios.

The advantage of GSEM becomes more pronounced under mixed disturbances (Fig. 2). As disturbance frequency increases from Low to High, the makespan improvement over MAPPO+Epi grows from 5.0% to 10.0%. Under high-frequency conditions, new disturbances arrive before previous ones are fully resolved, creating cascading disruptions. GSEM mitigates this by retrieving relevant recovery patterns from its memory pool.

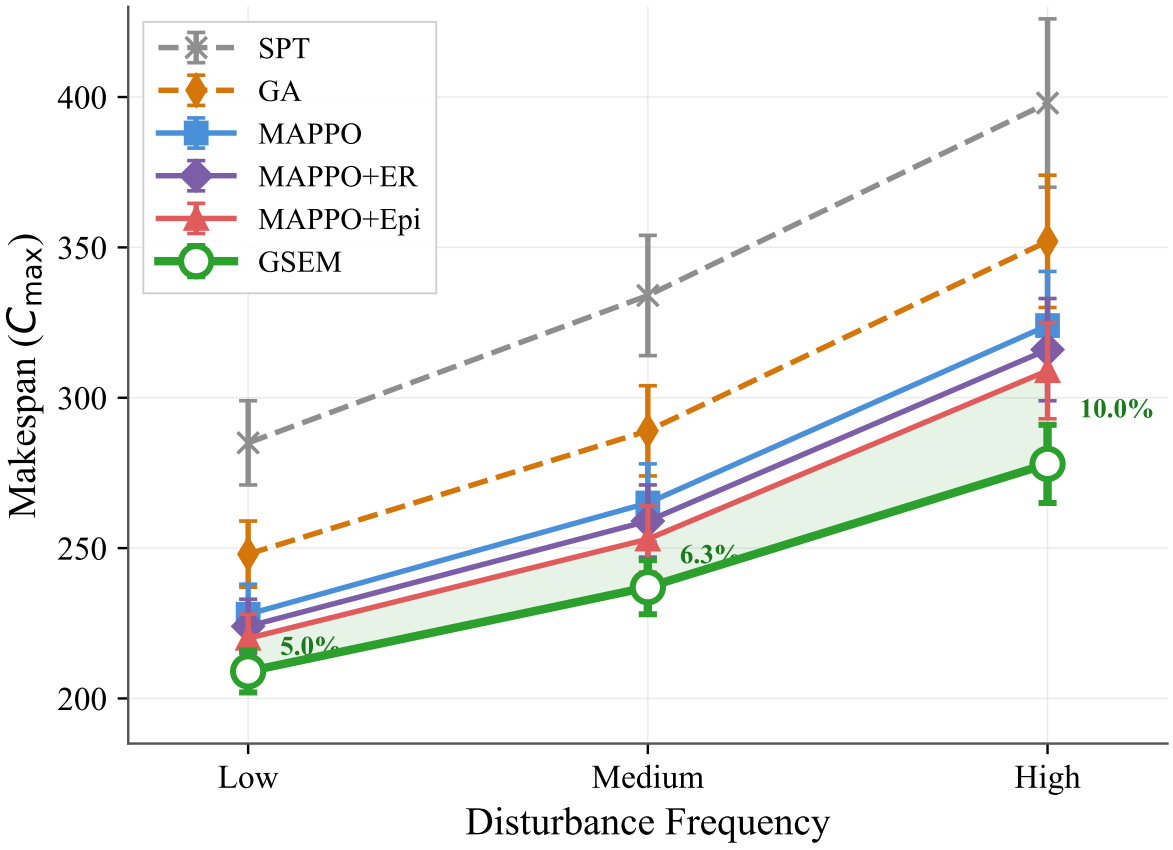


Fig. 2: Makespan under increasing disturbance frequency (mixed disturbances).

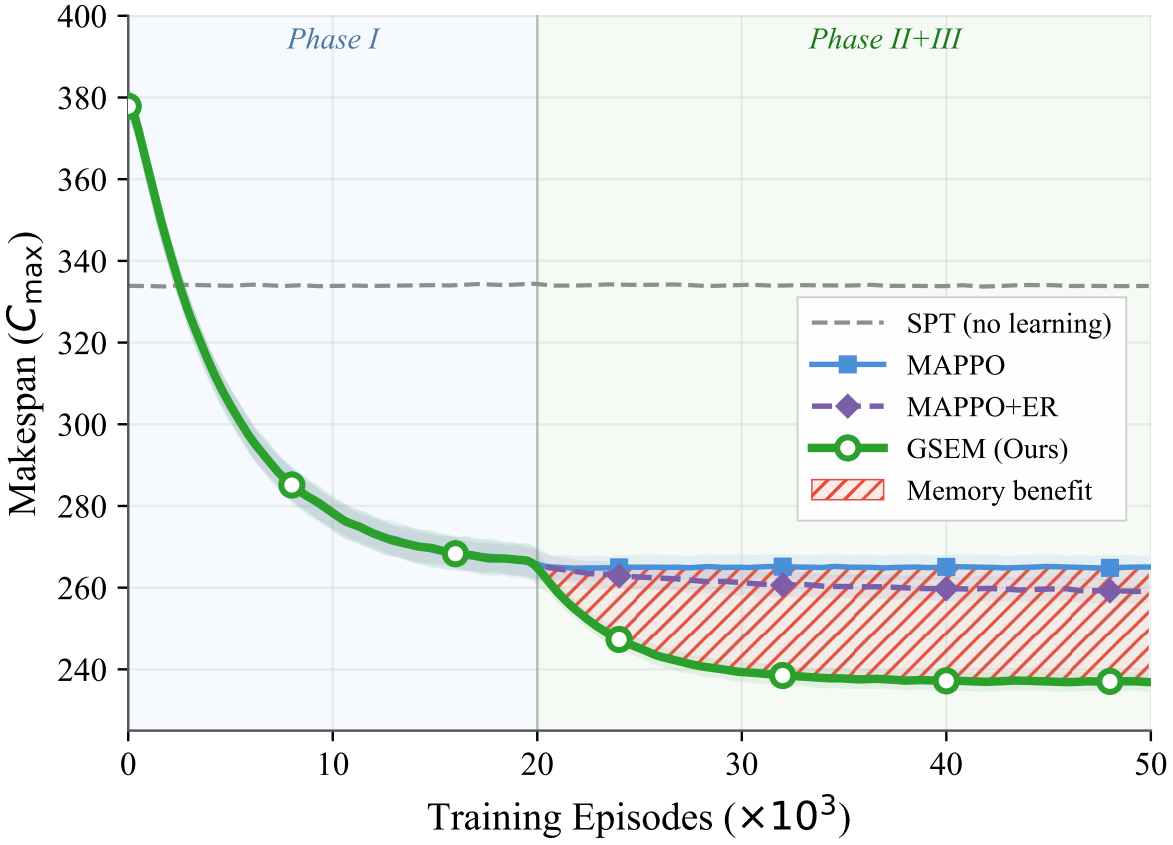


Fig. 3: Training curves under mixed disturbances (medium frequency).

IV-C *Training Convergence*

Fig. 3 presents the learning curves under mixed disturbances at medium frequency. In Phase I, all three methods exhibit similar convergence. Once Phase II+III begins, MAPPO and MAPPO+ER plateau, while GSEM continues to improve as the memory pool accumulates high-quality coordination episodes. This confirms that reusing past recovery structures provides a complementary learning signal that reactive policy optimization alone cannot capture.